Luleå Technical University
School of Computer Science, Electrical and Space Engineering
Luleå, Sweden

Final Report, Architecture Document
Group 14
D7065E, Embedded Intelligence at the Edge

FIRE DETECTION AND AUTONOMOUS RESPONSE

Ekpenyong Esu Ekpenyong
ekpekp-5@student.ltu.se

Examiner: Johan Kristiansson
LTU, Luleå

8th October 2026

Contents

1 1. Summary **1**

2 2. Use case and building context (BuildSim) **1**

3 3. Requirements **3**

4 4. Architecture **4**
4.1 4.1 Context diagram (C4 Level 1) 4
4.2 4.2 Container diagram (C4 Level 2) 5
4.3 4.3 Component diagram (C4 Level 3): inside the firelab service 6
4.4 4.4 Design decisions and justifications 8

5 5. Interfaces and communication **9**

6 6. Simulating the sensor values (the physical model) **11**

7 7. Autonomous services and data engineering **12**
7.1 7.1 The autonomous service 12
7.2 7.2 The data pipeline 14

8 8. Behaviour **15**
8.1 8.1 Key scenario, dynamic diagram 15
8.2 8.2 Process lifecycle, state machine 16

9 9. Deployment and component placement **16**

10 10. Test plan **17**

11 11. Evaluation and experiments **18**

12 12. Dashboard **21**

13 13. Risks and critical reflection **23**

1. Summary

This system watches an office building for fire and acts on what it finds without waiting for a person. Emulated smoke, carbon monoxide and temperature detectors observe the true state of each room imperfectly; a detector turns their readings into a probability of fire per room, and there are two of them to choose from, a hand-written fusion rule and a language model asked over the network; a per-room state machine climbs from NORMAL through INVESTIGATING and PRE-ALARM to CONFIRMED; and on confirmation the system releases the sprinkler, closes the fire door and sends the building out along routes that avoid the alarming rooms. The building, its rooms and its 3D view are BuildSim's. This project supplies the fire physics, the sensors, the intelligence and the response around it.

One principle runs through every decision below: **nothing downstream of the sensors is allowed to see the truth**. The physics knows the real temperature of every room. The detector, the agent and the interlocks only ever see what the sensors reported. The gap between those two views is what the whole project studies, and the dashboard shows the two side by side so that a false alarm or a missed fire is visible as a mismatch rather than hidden in a log.

Field	Value
Project title	Fire Detection and Autonomous Response
Team	Group 14, Ekpenyong Esu Ekpenyong
Use case	Multi-modal fire detection with a verified alarm, interlocked suppression and containment, and evacuation routing on a three-storey building
Repository	https://github.com/Ekpenyong-Esu/BuildSim-FireDetection
Version / date	v10.1 / 2026-10-08

Changes since the proposal. Three things changed, each because building the system taught something the proposal did not know.

- **One process instead of a dozen.** The proposal sketched nine services joined by MQTT, with ThingsBoard for storage and dashboards. Its own risk list said the hardest part would be keeping a dozen processes synchronised in simulated time. That risk turned out to be the whole problem: at forty times real time a reading published over a broker is stale before the subscriber sees it, and the physics, the sensors and the agent must agree on what "now" is for a dwell time to mean anything. I therefore built the system as one tick loop with four zones inside it, and the separation the proposal wanted from process boundaries is enforced instead by import rules that a test checks (Section 4.2). The trade-off is discussed in Section 13.
- **The second detector is a language model, not an isolation forest.** The proposal planned to compare a hand-written fusion rule with a trained anomaly detector. The rule alone caught every real fire and raised no false alarm at any noise level tested (Section 11), and the only data to train a forest on would have been the simulator's own output, so the comparison the proposal promised was made with a different second opinion: a language model behind an OpenAI-compatible endpoint, given the same readings the rule sees and nothing else, and asked for a probability, its evidence and a reason. It implements the same two-method protocol, so the agent, the interlocks and the dashboard did not change. Section 7.1 says how it is kept off the tick and Section 11 reports the comparison. The short version: the model catches what the rule catches, later, at a fraction of the speed, and without repeatability; the rule remains the default.
- **Containment against evacuation became an interlock, not a policy.** The proposal expected to tune a trade-off. In practice the safe rule is simple and should not be tunable: a fire door may not close on a room that is occupied or that lies on somebody's escape route, and doors never lock. The agent asks; the interlock decides; the request stands until it is safe to honour.

2. Use case and building context (BuildSim)

The building. The system runs on the A-building at Luleå University of Technology as BuildSim models it: three storeys, 956 named spaces, 1005 doorways and 32 stairwells, and seven real exits, all on the ground floor. BuildSim owns the floor plans, the walkable graph that joins rooms through their doors and the

stairwells that join the storeys, an equipment tree into which devices are registered, and a 3D viewer fed over WebSocket. BuildSim is not modified. It is instrumented and drawn on.

What BuildSim does not provide is anything that changes over time: there is no fire, no smoke, no people and no sensor physics. All of that is this project. firelab loads the floor plans once, turns them into a graph of rooms and door-gated couplings, and from then on writes back to BuildSim about once a simulated second: floor heat maps of the true temperature, smoke and CO, particle effects for fire, smoke and running sprinklers, the agent's alarm markers, fire-door state, every occupant's position, and each device's latest reading into the equipment tree. Two things are drawn into the viewer session rather than the building, because BuildSim keeps them per browser tab: the coloured outlines of alarming rooms and the escape route.

Actors. The **operator** watches the control panel, chooses what to set on fire, injects sensor faults, picks which detector scores the rooms, and can flip the system into supervised mode, where the agent still decides but a person has to press the button. The **occupants** are simulated people who influence nothing until the alarm sends them walking, and whose presence in a room is what stops its door being closed. Neither is on the control path. The loop from reading to sprinkler runs on its own.

The scenario the demo tells. A fast flaming fire starts in office A3505 on the top floor. Fifty-one seconds later the smoke reading has risen enough for the room to start INVESTIGATING; at 81 s it is in PRE-ALARM, and at 106 s, once the probability of fire has held above 0.75 for the confirmation dwell, the room CONFIRMS. In the same tick the sprinkler is released because the temperature reading corroborates the smoke, the fire door closes because the room is empty and on nobody's route, and all 130 people in the building start walking to a ground-floor exit along routes that avoid the alarming rooms. The last person is out 111 s after the alarm. Meanwhile the kitchen-cooking preset in the same room produces smoke and warmth for fifteen minutes and never gets past INVESTIGATING, because its CO never rises.

Why it matters. A smoke detector cannot tell a fire from burnt toast, a dusty corridor or its own drifting element. Real buildings are evacuated for all three, and the cost of that is that alarms get ignored. Telling them apart needs several modalities fused over time and across neighbouring rooms, which is a genuine job for an autonomous service. The response has a real conflict inside it too: closing a fire door holds the smoke back but takes away an escape route. Latency, robustness to broken sensors, and the safety of the actions drive the requirements that follow.

Where it sits. Fire detection is one of several systems that could run on the same building: ventilation, security and energy management are the others in the course. Each instruments BuildSim through the same API and each would be watched by the same facilities staff. Figure 1 shows the landscape; this document designs the highlighted system. One thing in it reaches off the laptop: when the language model is the chosen detector, the service asks a chat endpoint over HTTPS about one room at a time. It is optional. Without it, or when it is slow, the rule scores every room.

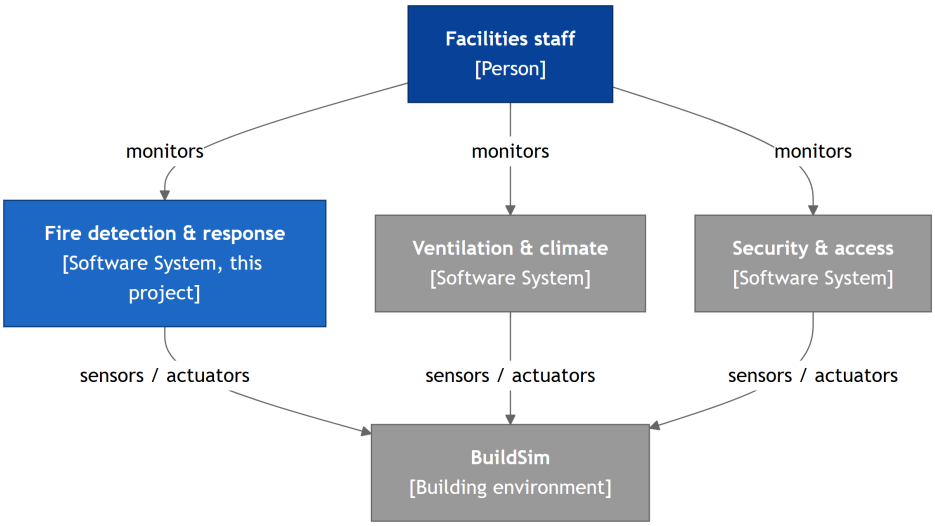


Figure 1: *C4 System Landscape. Several building systems share one BuildSim; the highlighted one is designed here.*

3. Requirements

Every requirement is a testable statement with a stable ID. Functional requirements (FR) say what the system does, safety requirements (SR) say what it must never do whatever the agent decides, and non-functional requirements (NFR) say how well. The last column names the test or experiment that verifies it; the test plan in Section 10 and the results in Section 11 close the loop.

ID	Requirement	Priority	Acceptance criterion	Verified by
FR-01	Confirm a flaming fire from sensor readings alone	Must	Room reaches CONFIRMED within 180 s of ignition	`test_agent`, `test_detector`, E1
FR-02	Confirm a smouldering fire, which has almost no heat	Must	CONFIRMED within 300 s (the scoreboard's miss limit)	E1
FR-03	Raise no alarm for cooking, dust, steam, or a drifting sensor	Must	Zero false alarms across all nuisance presets at sensor noise 0–3×	`test_scoring`, E1, E3
FR-04	On confirmation: release sprinkler, close the fire door, evacuate	Must	All three commands issued in the confirming tick and journalled with a reason	`test_agent`, E1
FR-05	Route every evacuee to a real ground-floor exit avoiding alarming rooms	Must	No route crosses an alarming room while a clear one exists; upper floors go down the stairs	`test_evacuation`, `test_walkways`, `test_multifloor`
FR-06	Explain every decision	Must	P(fire) is the sum of named contributions, or the model's verdict with its evidence and a one-line reason; every command and every refusal carries a reason	`test_detector`, `test_llm_detector`, dashboard
FR-07	Score the run against ground truth	Should	Detections, misses, false alarms, alarm delay and RSET reported live and exportable as CSV	`test_scoring`, `test_history`
FR-08	A second detector behind the same protocol, switchable while running	Should	The language model scores rooms from readings only; the agent, interlocks and panel are unchanged; the panel can switch back	`test_llm_detector`, `test_layering`, E7
SR-01	Water needs heat corroboration	Must	A sprinkler request is refused while the room's true temperature is below 35 °C	`test_interlocks`, E1
SR-02	Fire doors never lock	Must	A lock command is refused unconditionally	`test_interlocks`
SR-03	Never seal people in	Must	A close command is refused while the room is occupied or on an active escape route	`test_interlocks`, `test_publisher_doors`

ID	Requirement	Priority	Acceptance criterion	Verified by
NFR-01	Keep up with the clock	Must	One engine tick over the full building finishes inside the 250 ms tick budget at 1×; the breaking point at 40× is measured	E5
NFR-02	Survive a lying sensor	Must	A stuck, dropping-out or drifting smoke detector does not stop a flaming fire being confirmed	`test_sensing`, E4
NFR-03	Be repeatable	Must	The same seed gives the same run, tick for tick	`test_reset`, E1 (rerun)
NFR-04	Keep the truth out of the intelligence	Must	No module downstream of the sensors imports or reads the true room state	`test_layering`
NFR-05	Not flood BuildSim	Should	At most 24 sensor writes per publish; the viewer route and highlights are sent only when they change	`test_publishing_route`
NFR-06	Never wait for the model	Must	A tick never blocks on the network; a room with no fresh verdict is scored by the rule; a malformed answer is refused, not guessed at	`test_llm_detector`
NFR-07	Never outrun the model	Must	A verdict older than 60 simulated seconds is not used; the speed dial is capped at what measured answer times allow, and the journal says so	`test_pacing`, `test_llm_detector`, E7

There is a tension in this table. FR-01 and FR-02 want the alarm early; FR-03 wants it late enough to be sure; SR-01 to SR-03 may refuse the response the alarm asks for. The dwell times in the state machine resolve the first tension and the interlocks resolve the second, and the evaluation reports where each of those compromises costs something. NFR-06 and NFR-07 pull against each other too: a detector that answers eight seconds late is useless at forty times real time, so either the clock slows down or the rule takes over, and the system does the first and falls back to the second. The safety requirements apply whichever detector is installed; that is the point of keeping the interlocks outside the intelligence.

Traceability. The five architecture viewpoints map onto this document: context (Section 4.1), functional (4.2 and 4.3), information (Sections 5 to 7), behavioural (Section 8) and deployment (Section 9). Each requirement names the design that satisfies it and the test that proves it.

4. Architecture

4.1 Context diagram (C4 Level 1)

The context view fixes the boundary. Everything the system knows about the building comes through BuildSim's REST API, and everything it does to the building goes back the same way. The operator uses a browser. The occupants are simulated inside the system and are shown here as the people the system is for, because their presence is what the safety interlocks protect. There is one piece of external compute, and it is optional: the language model endpoint, which the service asks about one room at a time when that detector is selected. The proposal's overnight model training would have been the other candidate, and it was not needed. Nothing the system does to the building depends on the endpoint answering.

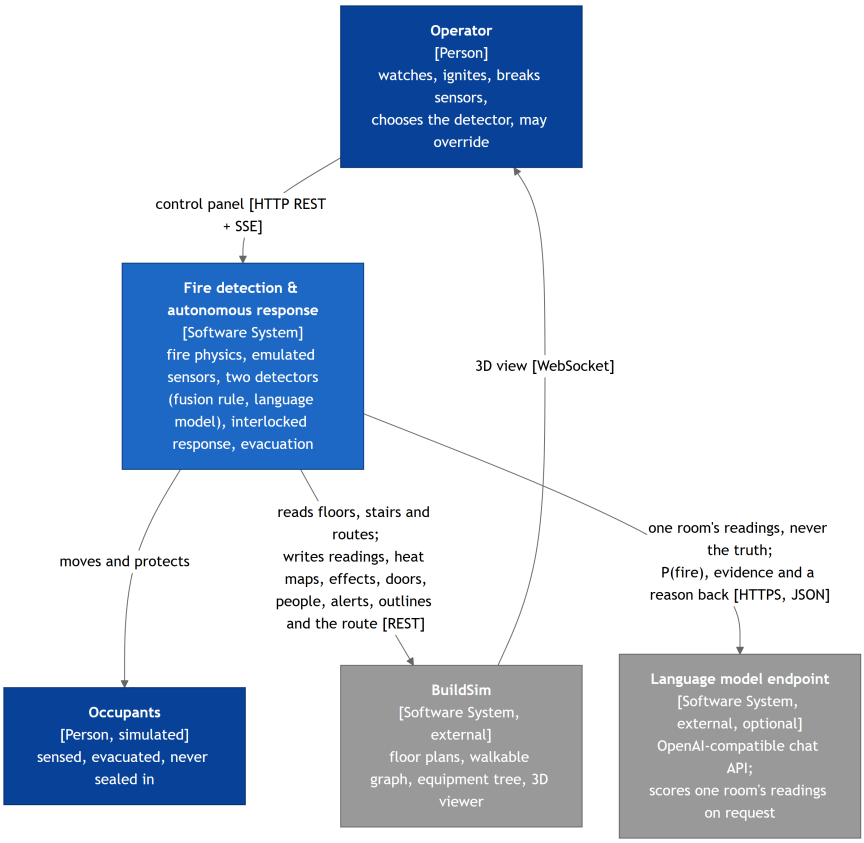


Figure 2: *C4 System Context (Level 1). The system in scope is blue; BuildSim and the language model endpoint are grey and external. Every arrow names its purpose and protocol. The model only ever receives readings, never the truth.*

4.2 Container diagram (C4 Level 2)

The container view opens the box. There are five separately running parts, and only one of them is this project's code. The fifth, the model endpoint, is the only one that is not on the laptop and the only one the system runs happily without.

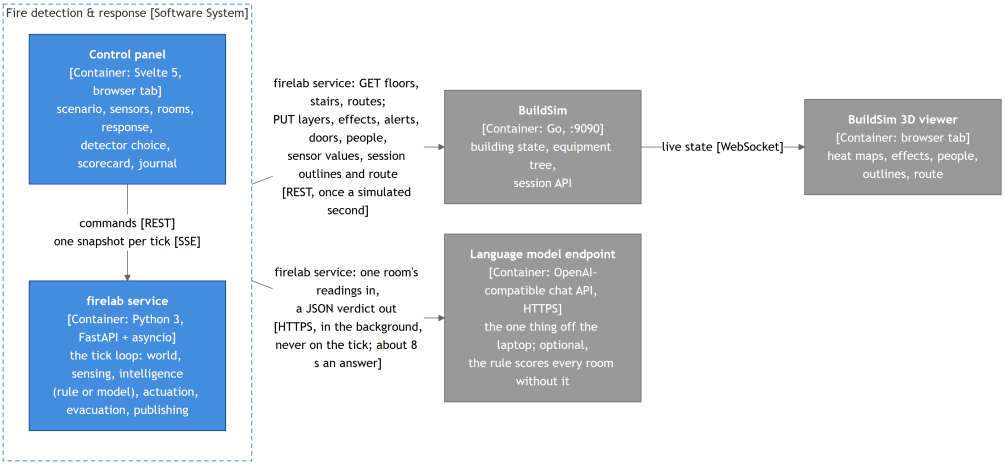


Figure 3: *C4 Container diagram (Level 2). The dashed boundary is the system; the grey containers are BuildSim's and the model endpoint. Every arrow carries its protocol. The call to the model is made by a background task, never by the tick.*

Why one service. The proposal's nine containers would each have held one of the four zones. I did not build it that way, for one reason: simulated time. The whole point of the system is to run a fire at forty times real time and still have a 25-second dwell mean 25 simulated seconds. Across a broker, every message would need a simulated timestamp, every subscriber would need to wait for every publisher before advancing, and a slow dashboard consumer could stall the physics. Inside one process the tick loop advances the world, samples the sensors, thinks and acts in a fixed order under one lock, and "now" is one number. The price is that the zones share a process, so a crash in one takes the others with it. Section 13 says what that costs and what would change if the system left the laptop.

What keeps the zones apart, since process boundaries do not, is the package layering: `domain` (pure, standard library only, no clock, no network), `adapters` (everything that speaks HTTP to BuildSim), `app` (the tick loop, the API and the snapshot). The dependency arrow only points inward, and a test, `test_layering`, fails the build if any module imports the wrong way or if anything in the intelligence path touches the true room state.

The control panel is a separate container because it has a different lifecycle: it is built once with Vite into static files, served by the service, and talks to it only through the REST endpoints and the server-sent event stream, the same interface a script would use. It has no knowledge of the engine's internals.

The model endpoint is outside the boundary for the opposite reason: it is not mine, it is shared with other users, and it answers in about eight real seconds when the tick has 250 milliseconds. Those two facts shaped the adapter in front of it. The service never waits for it, asks it about one room at a time, uses an answer for at most a minute of simulated time, and slows the clock down to what the measured answer time allows rather than letting verdicts go stale (Section 7.1). An OpenAI-compatible chat API was chosen because every hosted and local model server speaks it, so the endpoint, the key and the model name are three environment variables and nothing in the code names a vendor.

4.3 Component diagram (C4 Level 3): inside the firelab service

The service is the one container complex enough to need its own picture. Its components are the four zones of one tick, plus the things that sit around them. The three groups are the truth boundary: the components in the orange group may read the real state of the rooms, the components in the two blue groups may not, and the only thing that leaves the orange group is a sensor reading. The LLM adapter sits in the readings-only group on purpose. It speaks HTTPS, so by the layering rules it lives under `adapters`, but what it sends is the same feature vector the rule sees, and the layering test checks that it imports nothing that could tell it the truth.

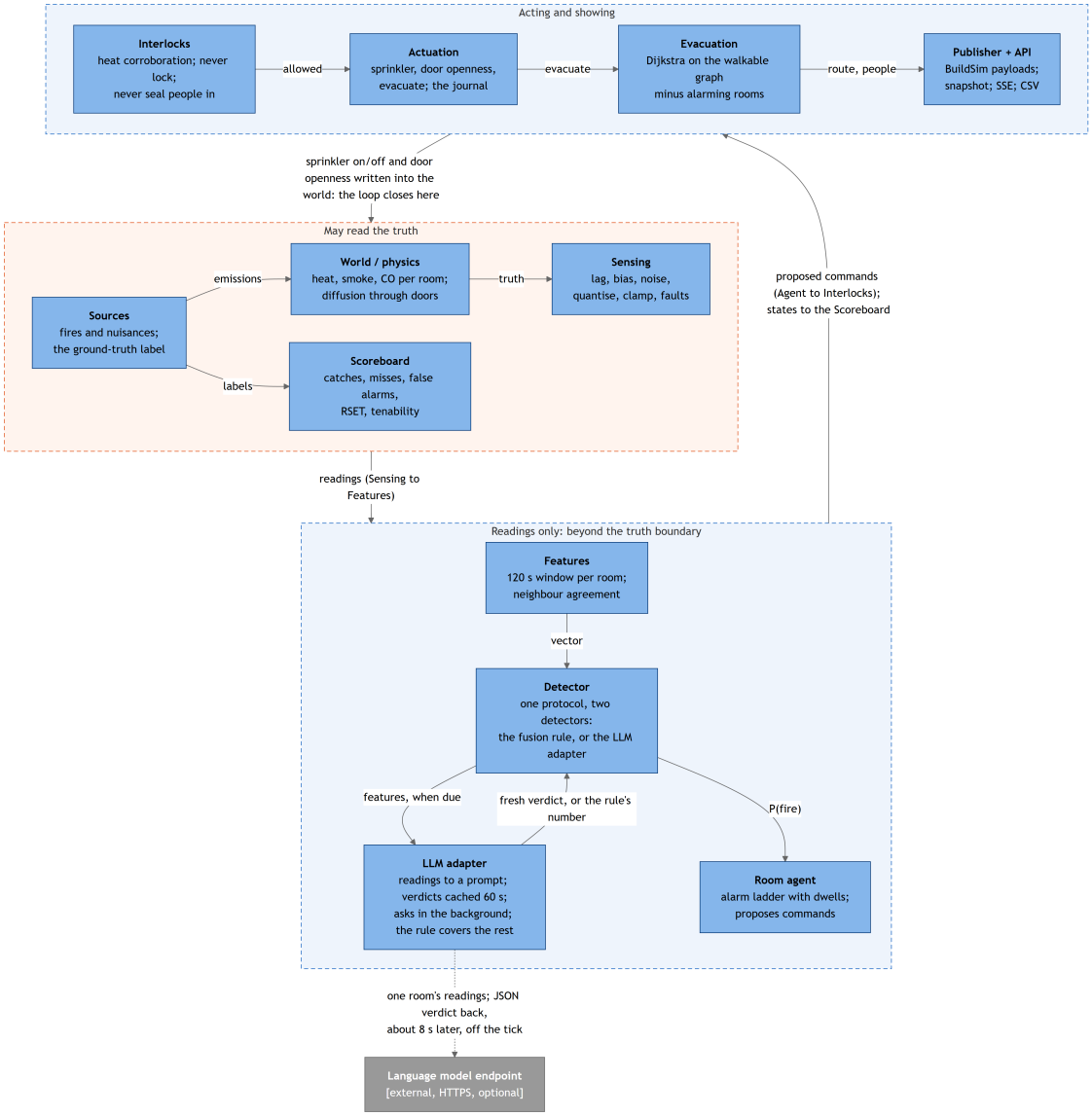


Figure 4: *C4 Component diagram (Level 3) of the firelab service. The four zones run in this order every tick: world, sensing, intelligence, actuation. Only readings leave the orange group, and the layering test fails if anything else does. The arrow into the orange group is the actuation zone writing sprinkler and door state into the world. The dotted arrow is the one call that leaves the process: the LLM adapter asking the model endpoint about a room, off the tick.*

Component responsibilities. Each component has one job, and the reason it is separate is the question the oral examination will ask.

Component	Responsibility (one job)	Module	Reason for separation
Sources	What a fire or nuisance emits over time; carries the ground-truth label	`domain/sources.py`	The label must never leak: only the scoreboard reads `kind`
World / physics	One lumped node per room; relax towards the source-driven equilibrium; diffuse through doors and stairs	`domain/physics.py`	Owns every time constant, so sources stay a few lines each
Sensing	Turn truth into a reading through the corruption chain; hold faults	`domain/sensing.py`	The truth-to-observation boundary; the one place noise and faults live
Features	A 120 s window per room; rate of rise, ratio, neighbour agreement	`domain/features.py`	Sees readings only; the detector's input contract

Component	Responsibility (one job)	Module	Reason for separation
Detector	Features → P(fire), with a contribution per feature; the fusion rule is one implementation	`domain/detector.py`	A protocol, so a second detector replaces the rule without touching the agent, and one did
LLM adapter	The same protocol, answered from a cache of the model's verdicts; asks in the background; the rule covers any room without a fresh verdict	`adapters/llm/`	The network must never hold the tick up; one file per job (prompt, verdict, policy, client, detector) so each is testable without an endpoint
Room agent	The alarm ladder with dwell times; proposes commands with reasons	`domain/agent.py`	Decides but never acts; testable with no actuators
Interlocks	Allow or refuse a command; say why	`domain/interlocks.py`	Kept outside the agent, so a mistrained agent still cannot trap anyone
Actuation	The only door between deciding and doing; writes sprinkler and door state into the world	`app/engine/actuation.py`	Closes the loop; a single audit point for every change
Evacuation	Route people around danger; walk them; choose the one route drawn	`app/evacuation.py`, `adapters/walkways.py`	The one domain-level job that needs a route lookup
Scoreboard	Detections, misses, false alarms, RSET, tenability	`domain/scoring.py`, `timeline.py`, `tenability.py`	Grades the run; the only consumer of the truth on the right-hand side
Publisher + API	Engine state → BuildSim payloads and the UI snapshot; REST, SSE, CSV	`adapters/publisher/`, `app/api/`, `app/snapshot/`	Presentation kept out of the loop; BuildSim's payload shapes in one place

4.4 Design decisions and justifications

Each row names the option taken, the alternative I considered, the requirement it serves and the price paid. A decision with no rejected alternative would only be an assumption.

Decision	Chosen	Alternatives	Justification (requirement)	Trade-off
Process model	One tick loop, four zones, one lock	One container per zone over MQTT (the proposal)	Dwell times and rates need one simulated clock; at 40× a brokered reading is stale on arrival (NFR-01, NFR-03)	No fault isolation between zones; the split is a future step
Zone separation	Import layering checked by a test	Trust the code review	The truth is one attribute away from the detector; a test makes restraint checkable (NFR-04)	A test to maintain as modules move
Detector	Hand-written fusion rule behind a protocol, as the default	Isolation forest; autoencoder; the model alone	Caught every fire, zero false alarms at every noise level, explains itself term by term, and costs nothing per room (FR-01, FR-03, FR-06)	Weights tuned by hand; no learning from data
Second detector	A language model over an OpenAI-compatible API, given readings only	Isolation forest trained on exported runs; no second detector	The comparison the proposal promised, without training on the simulator's own output; the verdict comes with evidence and a reason, so FR-06 still holds (FR-08)	About 8 s an answer, not repeatable, an external dependency
Model off the tick	Background worker, cached verdicts good for 60 s, rule fallback	Ask synchronously and wait; use stale verdicts	A tick has 250 ms and an answer takes seconds; a room must always have a number and a stale one is a wrong one (NFR-06, NFR-07)	Two detectors are in play at once, and the journal has to say which one is speaking
Pacing	Cap the speed dial at what measured answer times allow	Let the clock run and accept that verdicts expire	At 40× every answer arrived 320 simulated seconds late and the rule silently did all the work; a capped dial is honest (NFR-07)	With this endpoint the demo runs at 3× instead of 40×
Questions in flight	One at a time, most suspicious room first	Several at once	The endpoint answers requests one after another, so two in flight doubled the latency and halved the speed the clock could run at; the fire room gets its answer first	Quiet neighbours are mostly scored by the rule
Alarm logic	State machine with dwell times	Threshold on P(fire)	A single threshold flickers on noise; dwells cost seconds but stop doors cycling (FR-03)	Adds 45 s to the confirmation of a clear fire
Safety	Interlocks outside the agent	Rules inside the agent's policy	Whoever is wrong upstream, the building stays safe; the same gate applies to a person's click (SR-01 to SR-03)	A confirmed fire in an occupied room gets no door

Decision	Chosen	Alternatives	Justification (requirement)	Trade-off
Deferred commands	A refused request stands until honoured	Refuse once and forget	"Not yet" must not become "never"; the smoulder's sprinkler was refused at 201 s and released at 219 s once heat arrived (SR-01)	Re-checked every tick
Route search	Own Dijkstra over BuildSim's graph with alarming rooms removed	Ask BuildSim's router per exit	BuildSim only returns the shortest path, and a fire in a corridor makes every shortest path run through it (FR-05)	A second copy of the graph to keep consistent
Sensor emulation	Lag, bias, noise, quantisation, clamp, faults per device	Truth plus Gaussian noise	The interesting failures (saturation, stuck, dead) are not noise; the temperature clamp at 250 °C is what a real heat detector does (NFR-02)	More parameters to defend
Physics	Lumped node per room, first-order relaxation, door-gated diffusion	CFD; zone model with layers	Enough structure to make the loop close and the trends right, and cheap enough for 956 rooms 40 times a second (NFR-01)	Fire room overheats when sealed; no oxygen limit (Section 13)
Storage	In-memory rolling windows; CSV export	Time-series database; ThingsBoard	120 s is all the detector may see and 20 minutes is all the charts need; a database was a process to keep in step with the clock (NFR-01)	History is lost on restart; long runs are exported, not stored
UI transport	Server-sent events, one snapshot per tick	Polling; WebSocket	One-way push with automatic reconnect, and a plain HTTP stream any tool can read	Snapshot rebuilt every tick even with no listener
BuildSim writes	Once per simulated second, complete collections, 24 sensor values per publish	Every tick; deltas	BuildSim replaces a whole collection per write; too many writes buried it (NFR-05)	Sensor labels in the tree lag behind the truth

5. Interfaces and communication

Four patterns are used, each where it fits. **REST** carries every command and every call to BuildSim, because each is a one-to-one request whose result the caller must know: a refused command comes back with the interlock's reason. **Server-sent events** carry the snapshot to the control panel, because that is one-way, high-rate and needs no reply. **WebSocket** is BuildSim's own channel to its viewer and is used, not built. **HTTPS chat completions** carry one room's readings to the language model and a JSON verdict back; the call is made by a background task with a 30 s timeout and one retry, and its result is a cache entry, never a tick. There is no pub/sub, for the reason given in Section 4.2.

Naming. A room is addressed everywhere as `level/name`, for instance `level2/A3505`, because a bare name is not unique across storeys. A device is `fl-<level>-<room>-<modality>`, so its room and kind can be read off its ID. Units are fixed and never carried in messages: temperature in °C, smoke as optical obscuration in 1/m, CO in ppm, distances on the plan in half-metre units, and simulated time in seconds since midnight.

The control-panel API. Every request body is a typed model; unknown keys are rejected by the framework before the engine sees them.

Method	Path	Body	Semantics
GET	`/api/state`		The full snapshot, once
GET	`/api/events`		SSE: one snapshot per tick, 4 Hz
GET	`/api/rooms`		Every space for the pickers
GET, PUT	`/api/config`	partial config	Live settings: speed, thresholds, dwells, noise, population; `detector.kind` swaps the detector on the next tick
POST	`/api/clock`	`{seconds?, factor?, running?}`	Play, pause, jump, 0.1–600×
POST	`/api/scenario/ignite`	`{space, kind, growth, delay, peak_kw}`	Schedule a source; `kind` is also the label
POST	`/api/scenario/preset`	`{preset, space}`	Instrument the room and its neighbours, then ignite
DELETE	`/api/scenario/{id}`		Put a source out; what it emitted still disperses

Method	Path	Body	Semantics
POST	`/api/sensors/deploy`	`{spaces, modalities}`	Install devices; a room without devices is unmonitored
POST	`/api/sensors/fault`	`{device_id, fault}`	`stuck | dropout | drift | dead | none`
POST	`/api/agent/mode`	`{auto}`	`false` is supervised: decisions shown, not carried out
POST	`/api/actuators/command`	`{kind, space, value}`	A manual command; passes the same interlocks; returns `{allowed, reason}`
GET	`/api/history?space=`		The last 240 samples of one room
GET	`/api/export.csv`		Truth beside reading beside P(fire), every room, labelled
POST	`/api/reset`, `/api/reload`		Quiet building; reload floors from BuildSim

The snapshot is the one data structure the panel reads. Each monitored room appears as below, and the three columns of the Rooms table are the three objects in it: what is true, what was reported, and what the system believes.

```
{"key": "level2/A3505", "level": "level2", "name": "A3505", "kind": "room",
 "truth":   {"temperature": 648.0, "smoke": 0.668, "co": 107.0},
 "reading": {"temperature": 250.0, "smoke": 0.665, "co": 101.0},
 "p_fire": 0.994, "state": "CONFIRMED", "tenable": false,
 "sprinkler": true, "door": "closed", "occupants": 0,
 "devices": [{"id": "fl-level2-A3505-smoke", "modality": "smoke",
              "fault": "none", "reading": 0.665}, ...]}
```

The temperature reading of 250 against a truth of 648 is not a bug in the example. It is the heat detector's range clamp, and it is the kind of thing the snapshot exists to make visible.

The BuildSim interface. Truth and belief are published to different endpoints so that a false positive shows on screen as a room glowing red on the belief layer while cool on the heat map.

Endpoint	Direction	Carries	When
`GET /api/building/floors/{level}`	read	rooms, polygons, walkable graph	once, at load
`GET /api/building/cross-floor-edges`	read	stairs and lifts joining storeys	once, at load
`GET /api/graph/route`	read	a shortest path, multi-floor	fallback when no clear route exists
`PUT /api/room-layers`	write	true temperature, smoke, CO; and P(fire) as its own layer	every simulated second
`PUT /api/effects`	write	fire, smoke and sprinkler particles	every simulated second
`PUT /api/alerts`	write	the agent's alarm markers with P(fire)	every simulated second
`PUT /api/doors`	write	fire-door state; also blocks the walkable router	every simulated second
`PUT /api/entities`, `/api/occupancy`	write	every occupant's position and figure; head count per room	every simulated second
`PUT /api/sensors/{id}/value`	write	one device's latest reading, into the equipment tree	24 per publish, round robin
`PUT /api/sessions/{id}/highlights`	write	red, orange, yellow outlines by alarm state	when they change
`PUT /api/sessions/{id}/route`	write	the one escape route drawn, each node with its storey	when it changes

A route is a list of waypoints, each carrying its level so that a path down the stairs is drawn on the storeys it crosses, plus its length in metres. A closed door is published to BuildSim as well as applied in the physics, so that BuildSim's own router, which the viewer's route finder uses, stops sending people into a sealed room.

The model interface. One request per room, `POST /chat/completions` on whatever base URL is configured, with the fixed system prompt and the room's readings as plain lines with units. The prompt is the whole contract: it tells the model it sees only the sensors, never the room, and it lists what separates a fire from a nuisance in the same terms the rule uses. The reply must be one JSON object:

```
{"p_fire": 0.92,
 "evidence": [{"name": "CO 95 ppm", "weight": 0.30},
              {"name": "smoke rising", "weight": 0.25}, ...],
 "reason": "Smoke, CO and temperature are rising together."}
```

Anything that does not parse as that, or whose probability is not a number, is refused rather than repaired, and the room stays with the rule. The evidence becomes the "Why" bars on the panel, alongside a bar for the verdict itself on the rule's logit scale, so the two detectors read the same way on screen.

6. Simulating the sensor values (the physical model)

BuildSim provides the rooms but nothing that changes, so the fire, the air in every room, the sensors and the people are all simulated here. The aim was never physical fidelity. It was a model with enough structure that a decision made on its readings means something: fires that keep growing, nuisances that peak and fade, smoke that leaves through doors and climbs stairwells, and, above all, actuator commands that change the next reading so that the loop closes.

Sources. A source is a scripted event with a kind, a room and a start time. The kind is also the ground-truth label the scoreboard uses, and nothing downstream of the sensors may read it. Each kind declares what it emits at time e since ignition, as a heat release rate and the smoke and CO the room would settle to if nothing else happened:

Kind	Label	Emission	What makes it hard
flaming	fire	$q = \min(\alpha e^2, q_{\max})$ with the standard t^2 growth coefficients (slow 0.0029 to ultrafast 0.1876 kW/s^2); smoke $0.06\sqrt{q}$, CO $4\sqrt{q}$	The easy case: everything corroborates
smouldering	fire	3 kW, smoke 0.9 1/m and CO 900 ppm, all ramping with $1 - e^{-e/300}$	Almost no heat, so the sprinkler interlock refuses water
cooking	nuisance	12 kW, smoke 0.35, CO 25 ppm, as a bump that rises over 3 min and fades over 15	Warm and smoky, but the CO never comes
dust	nuisance	smoke 0.5, a 20 s bump; no heat, no CO	Fools a smoke detector on its own
steam	nuisance	smoke 0.7, a 30 s bump; no heat, no CO	As dust, denser and shorter

The shape over time is what matters. A real fire keeps getting worse; a nuisance grows, holds and fades. That difference over time, together with the ratio of CO to smoke, is what the detector has to pick up.

Room physics. Every space is one lumped node with a true temperature T, smoke obscuration S and CO concentration C. Each step of Δt seconds has two phases. First each room relaxes towards where its own source is pushing it:

$$T \leftarrow T^* + (T - T^*)\, e^{-\Delta t/\tau_T}, \qquad T^* = T_{\text{out}} + \min\Big(140\,(q/V)^{2/3},\, 900\Big),$$

with the same first-order approach for smoke and CO towards their equilibrium values, and time constants $\tau_T = 90$ s, $\tau_S = 45$ s, $\tau_C = 60$ s. V is the room's volume, its floor area from the plan times a 3 m ceiling, so a lecture theatre dilutes a fire and a cupboard does not, and the sub-linear rise with heat density is capped at post-flashover temperatures. Then neighbouring rooms trade what they hold through every doorway and stairwell:

$$\Delta X_{a\to b} = \min\big(g\,k_X\,\Delta t,\, 0.5\big)\,(X_a - X_b), \qquad g = \text{conductance} \times \text{openness},$$

for $X \in \{T, S, C\}$ with diffusion rates $k_S = 0.35$, $k_C = 0.30$, $k_T = 0.12$. The conductance of a doorway comes from BuildSim's walkable graph and is 0.6 for a stairwell, which is how smoke climbs floors. The share a room gives away in one step is capped across all of its doorways together, because a corridor with twenty doors would otherwise hand out several times what it holds and the building would fill with smoke nobody produced. That was a real bug, found by running the building at high speed.

Closing the loop. Two actuator states live inside the physics. An active sprinkler multiplies the room's heat release by 0.25 and its smoke equilibrium by 0.4. A closed fire door sets the openness of every coupling

into that room to 0.15, because a real fire door leaks. Both are written by the actuation zone and read by the next physics step, so a command issued at tick n changes the reading at tick $n+1$. Section 11 measures what that buys.

Sensors. A reading is never the truth. Every device applies the same chain, in this order, each step with its own per-modality parameters:

`lag → calibration bias → noise → quantisation → clamp → fault`

Modality	Lag τ	Noise σ	Quantum	Range	Interval	Unit
smoke	12 s	0.010	0.005	0–3	5 s	1/m
CO	25 s	3.0	1	0–2000	5 s	ppm
temperature	30 s	0.15	0.1	−20–250	5 s	°C

Temperature lags most because a heat detector has thermal mass. The clamp at 250 °C is why the dashboard shows a reading of 250 against a truth of 648 in a developed fire: the detector is saturated, as a real one would be. Noise is scaled by one live setting, 0 for a perfect instrument and 3 for a badly behaved one, and the evaluation sweeps it. A device can also be given one of four faults: *stuck* repeats its last value, *dropout* skips 40 % of samples, *drift* adds an error that grows without bound, and *dead* reports nothing at all. Faults are injected from the Rooms table and are what the two hardest presets are built on.

People. The proposal's daily occupancy schedule was not built, because nothing in fire detection depends on how many people are in a room at nine in the morning. What matters is where people are when the alarm sounds, so the population is scattered once over the routable rooms with a seeded generator: by default 100 students, 10 lecturers, 5 staff, 5 security guards and 10 visitors, 130 people in all. Each role walks at its own multiple of the configured 1.3 m/s. The difference is familiarity rather than fitness: a guard who has drilled the routes is 15 % faster than a student, a visitor who has never been here 15 % slower. Once evacuating, a person follows the waypoints of their route, carrying with them a fractional effective CO dose accumulated from every room they pass through, after ISO 13571; walking out through a smoke-logged corridor costs, and keeps costing afterwards. A room stops being escapable at 60 °C or when smoke reduces visibility below 10 m, and the scoreboard counts those rooms as lost.

7. Autonomous services and data engineering

7.1 The autonomous service

The intelligence runs once per tick over every monitored room, in three steps that each see only the output of the one before: readings become features, features become a probability, and the probability drives a state machine. The middle step has two implementations, a hand-written rule and a language model, and the operator can switch between them while a fire is burning. The rule is the simplest thing I found that meets the requirements; the model is there to find out what a second opinion costs and buys, and the end of this section says how it is kept from slowing the loop down.

Features. Each monitored room keeps a rolling 120 s window of readings per modality. From it come the latest smoke, CO and temperature; the smoke rate of rise per minute; the temperature rise above the building's 20 °C baseline; the ratio of CO to smoke, which is high for a smoulder and near zero for dust or steam; the share of monitored neighbours that also see smoke; and the coverage, how many modalities reported at all. Nothing here reads a room's true state, and the test suite fails if that ever changes.

The rule. The default detector is a hand-written fusion rule. Each feature contributes a bounded, weighted term, the terms are summed with a negative bias, and the sum passes through a logistic function:

$$z = -3.4 + 3.2\,\min(S, 1.5) + 1.8\,\tfrac{\max(C-25,\,0)}{125} + 1.6\,\min\!\left(\tfrac{C/S}{250},\,2.5\right)\gamma + 0.55\,\tfrac{\min(\Delta T,\,60)}{10} + 1.4\,\dot{S} + 0.8\,N,$$

$$P(\text{fire}) = \sigma(z)\cdot\left(0.55 + 0.45\,\tfrac{\min(\text{cov},\,3)}{3}\right),$$

where $\gamma = \min(S/0.15, 1)$ gates the ratio until there is smoke to divide by, $\dot{S}$ is the clipped smoke rate, N the neighbour agreement, and the last factor holds the score back when fewer than three modalities are reporting. CO starts counting only above the 25 ppm that cooking peaks at, which is what separates a frying

pan from a smoulder. Every term is bounded so that no single modality can saturate the sum on its own, which is why a dead smoke detector does not blind the system and why a runaway one does not alarm it (Section 11).

The detector is a protocol with two methods, `probability` and `explain`, and the second is not optional. The explanation is the calculation: the contributions are the terms above, the dashboard draws them as bars under the heading "Why 99 %", and a detector that could not say why would be one nobody could sign off. The model has to implement the same two methods, and does.

The alarm ladder. The agent is one state machine per room, driven only by the probability and the clock:

Transition	Condition	Dwell
NORMAL → INVESTIGATING	$P \geq 0.35$	none
INVESTIGATING → PRE-ALARM	$P \geq 0.55$ held	20 s
PRE-ALARM → CONFIRMED	$P \geq 0.75$ and in pre-alarm for	25 s
CONFIRMED → SUPPRESSED	sprinkler running and $P < 0.75$ for	25 s
any alarm → CLEARING	$P \leq 0.25$ held	90 s
CLEARING → NORMAL	quiet for	90 s

The dwells are the false-alarm defence. A single threshold would flicker on sensor noise and open and close fire doors all day; holding a level for a fixed time costs 45 s on a clear fire and buys silence on cooking, dust and drifting sensors. Standing down takes far longer than raising the alarm did, and that is intended. The thresholds and dwells are live settings, so the trade-off can be explored from the panel.

What it asks for. Only two states issue commands. Entering CONFIRMED asks for the sprinkler on, the fire door closed and the building evacuated; entering CLEARING asks for the reverse. Every command carries a reason string, "P(fire)=0.91 in A3505", that goes into the journal. The agent never writes actuator state. Its requests go to the interlocks, and a request the interlocks refuse stays pending and is offered again every tick, because "not yet" must not become "never". In the smouldering run the sprinkler was refused at 201 s for lack of heat and released at 219 s when the room crossed 35 °C, and nobody had to ask twice.

Failure and safety. If the intelligence is wrong, the interlocks hold: no water without heat, no locked door ever, no closed door on an occupied room or an escape route. If it is unavailable, nothing moves: actuators keep their last state and the building is as safe as it was without the system. If a person disagrees, supervised mode shows every decision and carries out none, and a manual command from the panel passes through the same interlocks and is journalled with the same reasons. With no working sensor in a room, the detector has no opinion rather than a low one, so an uninstrumented room is never marked safe. None of this changes when the model is the detector, which is the reason the interlocks were kept outside the intelligence in the first place.

The second detector: a language model. The proposal planned to compare the rule with an isolation forest. I built the comparison with a language model instead, for two reasons. The only labelled data to train a forest on would have been this simulator's own output, so the forest would have learned my physics rather than fires. And a model that answers in words can be asked to explain itself, which is the requirement a trained detector usually fails. The adapter in `adapters/llm/` implements the same two-method protocol as the rule and sends one room's readings, the same features the rule sees and nothing else, to any OpenAI-compatible chat endpoint. The system prompt tells the model it sees only the sensors, never the room, and lists what separates a fire from a nuisance in the same terms the rule encodes: a flaming fire gets worse in all three modalities, a smoulder is dense smoke and high CO with no heat, cooking is warm and smoky with CO under 25 ppm, dust and steam are smoke alone, a lone sensor that disagrees with its neighbours is probably broken. The reply is one JSON object with a probability, a list of weighted evidence and a one-sentence reason, parsed strictly: anything malformed is refused, not repaired.

The engineering is in what happens around the call, because the tick has 250 ms and the endpoint I had takes about eight seconds. Figure 5 shows the arrangement. `probability()` never waits. It returns the model's last verdict for the room if that verdict is younger than 60 simulated seconds, and the rule's number otherwise, and at most queues a question. A worker task beside the tick loop sends the queued questions one at a time, the most suspicious room first by the rule's own score, at most once per room every ten simulated seconds and never for a room whose sensors show nothing. A quiet building costs nothing. A verdict is stamped with the time the readings it saw were taken, not the time it arrived, which is what makes the 60 s expiry

mean something. The journal records each hand-over in both directions, every verdict that moves the score by a tenth, and the model's reason with it.

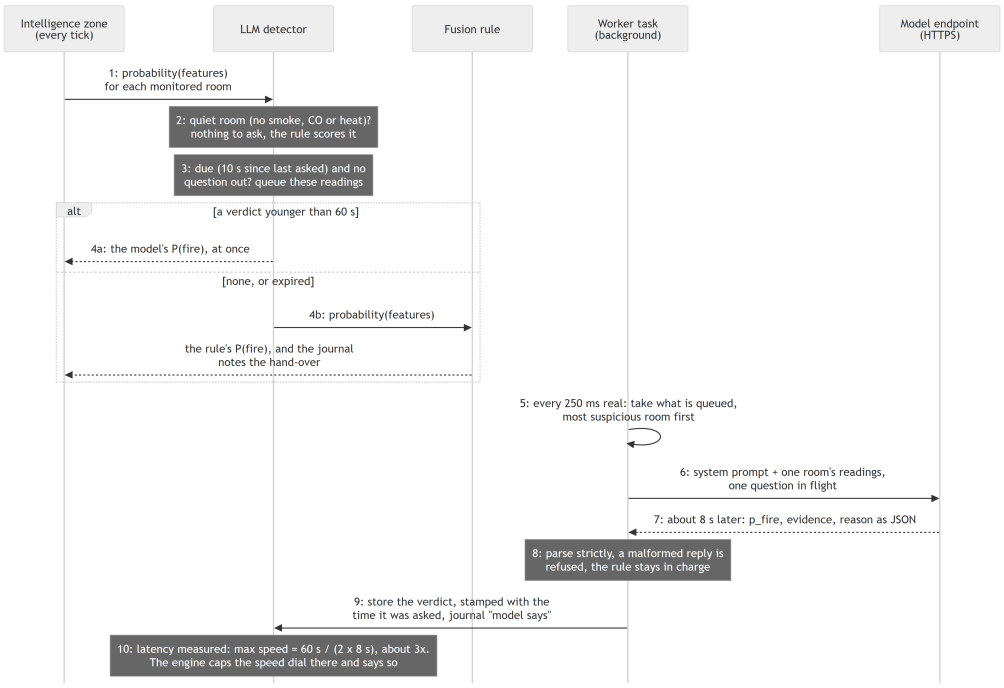


Figure 5: *How the language model scores a room without holding the tick up. The tick only ever reads a cache or the rule; the worker fills the cache in the background; a measured answer time of about 8 s caps the clock near 3× real time.*

Pacing. The first time I ran the model at forty times real time, nothing happened: every answer arrived 320 simulated seconds after it was asked for, every verdict had expired on arrival, the rule scored every room, and the panel said "llm". That is the failure mode to design against, and the fix is not a longer expiry. A verdict a minute old is a verdict about a room that has since doubled its smoke. Instead the detector measures how long answers take and reports the largest speed at which a verdict still arrives with half its life left, $60/(2 \times \text{latency})$; the engine caps the speed dial there every tick and writes one journal line saying so. With the endpoint used here that is $3\times$. The endpoint answers requests one after another, so asking about two rooms at once only doubled the latency and halved that cap, which is why one question is in flight at a time and the fire room gets it. Switch back to the rule and the cap lifts.

What the comparison is for. The rule is the baseline the model has to beat on three counts: whether it catches the same fires and ignores the same nuisances, how much later, and whether its "Why" is as auditable as a sum of terms. Section 11 reports all three. The model's evidence is drawn as the same bars, with one extra bar for the verdict itself on the rule's logit scale, so the two are judged on the same screen.

7.2 The data pipeline

Readings flow through a short in-memory pipeline that serves four consumers at once: the detector, the control panel, the scoreboard and the CSV export. It is small. A time-series database and a broker were in the proposal, and were dropped because every one of them would have been another process to keep in step with a clock running forty times faster than the wall.

- **Ingestion.** Every device samples the truth at its own interval and appends the reading to its room's window. The window keeps 120 s of simulated time per modality, because that is all the detector is allowed to look at.
- **Features and score.** Once per tick the features are extracted, the detector scores the room, the contributions are kept for the "Why" panel, and the agent steps.

- **The flight recorder.** Every 5 simulated seconds, one row per monitored room: time, true temperature, smoke and CO, the three readings, and P(fire). Each room keeps its last 240 rows, about twenty minutes, which is what the charts show.
- **The journal.** Every state change, every command with its reason, every refusal with the interlock's reason, and every detector verdict that moved a room's score by a tenth, with the rule's top three terms or the model's reason; the last 200 lines. This is where the lessons show up.
- **The scoreboard.** Judged once per new alarm against the source labels: a catch with its latency, a miss after 300 s unconfirmed, or a false alarm with the nuisance that caused it. Smoke that came through the door from a real fire next door is not a false alarm; a fire on another floor is no excuse.
- **Out.** The snapshot is rebuilt once per tick and pushed to every open panel over SSE. The whole recorder, labelled, is one GET away as CSV.

Data quality is handled at the source rather than cleaned afterwards, because the project exists to see what the intelligence does with bad data. A dead device stops contributing and the coverage factor lowers the score; a dropout leaves gaps the window tolerates; a stuck value flattens the rate term. Retention is short by design. Anything longer than twenty minutes is exported, not stored, and a restart starts clean.

8. Behaviour

8.1 Key scenario, dynamic diagram

Figure 6 is a C4 dynamic diagram of the office fire from Section 2, with the times measured in the run reported in Section 11. It shows what the static diagrams cannot: the tick order, the dwells, and the moment the loop closes.

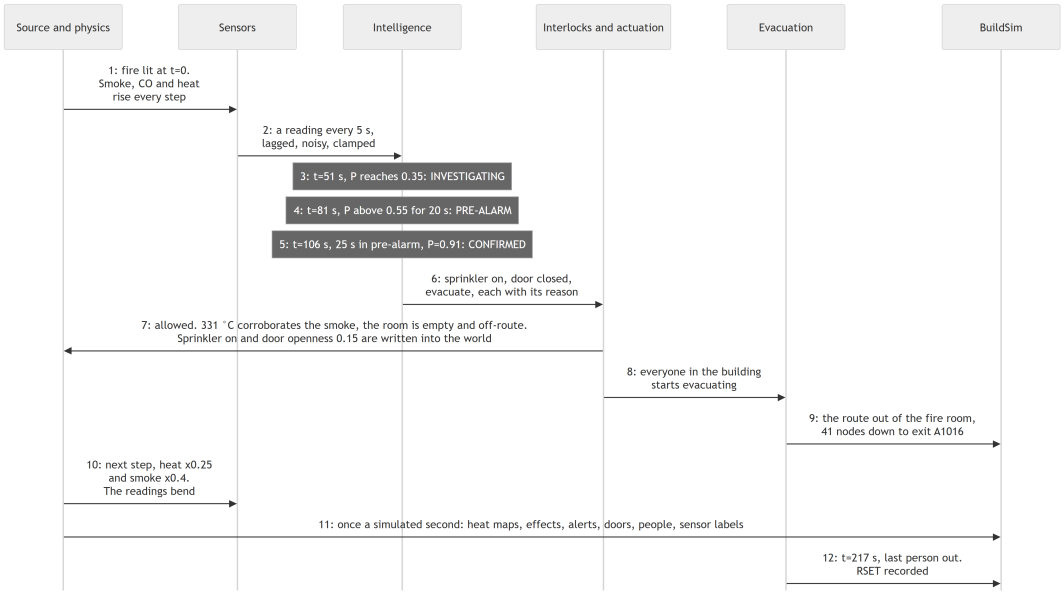


Figure 6: *C4 Dynamic diagram of the office fire, with the times measured in Section 11. Steps 3 to 5 are the dwells; step 7 is where the interlocks decide; step 10 is the loop closing.*

Two things in the order matter. The response is issued and applied in the same tick that confirms, so nothing waits for the next reading. And the fire door closed only because A3505 happened to be empty and on nobody's route; had a student been inside, step 7 would have refused the door, journalled why, and kept the request pending until they left.

8.2 Process lifecycle, state machine

Every monitored room has one of these, and its transitions are the code in `domain/agent.py`. The dwells are on the arrows.

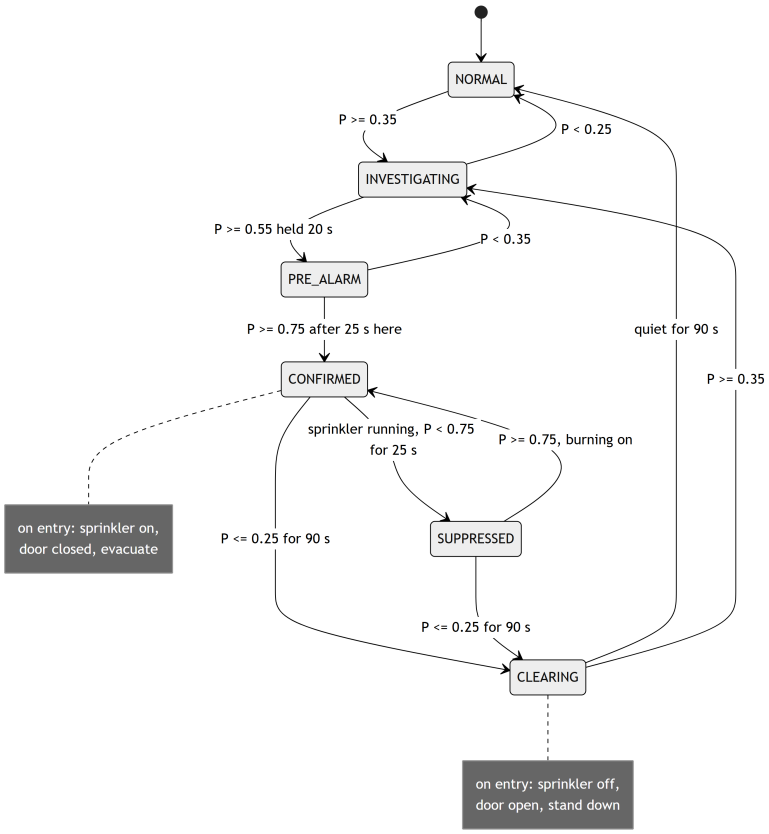


Figure 7: *The per-room alarm state machine. Only two states ask for anything; every other arrow only changes what the room believes.*

SUPPRESSED is still an alarm: the fire is being held, not out, and the confirmed state's outstanding requests stand. A fire that burns on through the water climbs straight back to CONFIRMED. Standing down needs the probability to stay at or below 0.25 for 90 s and then a further 90 s of quiet before the room stops being flagged as an incident, so a sensor that recovers for a moment cannot reopen a door onto a fire. The process itself has a lifecycle of its own outside this diagram: on startup the service loads the floors from BuildSim, installs the detector the configuration names, starting the model's worker task if it is the model, and scatters the population; if BuildSim stops answering, the service marks itself disconnected, keeps simulating, stops publishing, and waits for a reload rather than crashing. Changing the detector while running stops one worker and starts the other under the tick lock, so no tick sees half a swap, and the rooms' alarm states carry over untouched because the agent never knew which detector it was listening to.

9. Deployment and component placement

Everything that matters runs on one laptop, so deployment is about how the system is composed from processes rather than where in the world they are. There are two processes and two browser tabs, and they talk only through the interfaces of Section 5. One optional call leaves the machine: the language model, when it is the chosen detector, is a shared server on the university network.

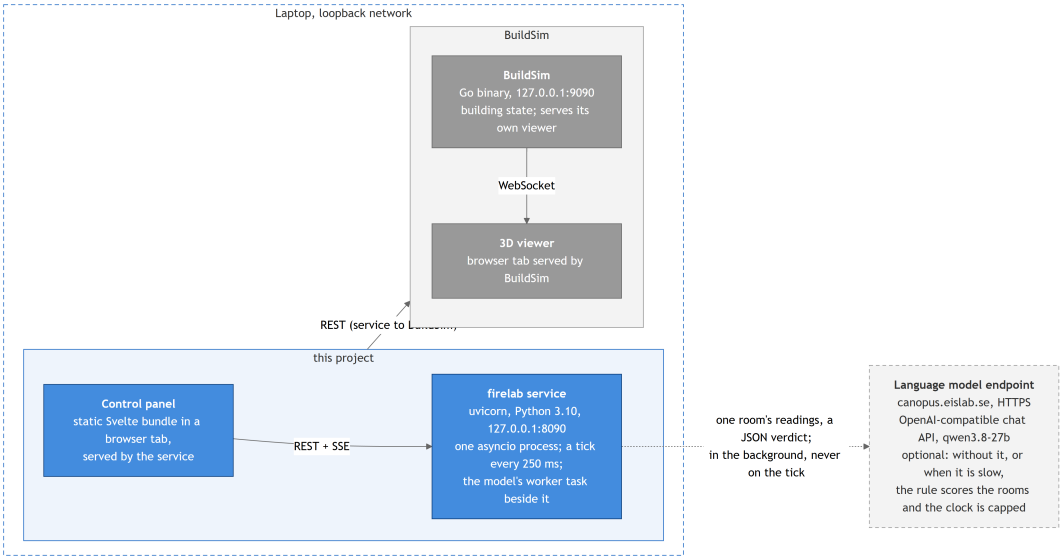


Figure 8: *Deployment view. Two processes, two tabs, loopback only. The dotted box is the one thing that leaves the edge, and the system keeps detecting without it.*

Why this placement. The control loop, from physics through sensors to the interlocks and back, lives in the firelab process and touches nothing outside the machine except BuildSim over loopback. In a real building that process is the edge node: it must keep working when the internet does not, and it must answer inside the tick budget, which Section 11 measures. The model endpoint is the test of that principle. It is off the edge, it is slow, and it is somebody else's, so the adapter treats it as advice rather than as a dependency: the rule scores every room the model has not answered for, the tick never blocks on the call, and if the endpoint disappears the journal says so and nothing else changes. That is the design a real building would need for any cloud service on the detection path, and it is why the box is dotted. BuildSim stands in for the building's own controllers and is left alone.

What is lost by merging the zones into one process, since the course asks: fault isolation and independent restart. If the physics throws, the agent dies with it. That is acceptable for a simulator on a laptop and would not be for a building. The mitigation built in is the layering, which keeps the zones as separable as separate processes would, and the seam is known: the sensing zone already produces per-device readings keyed by device ID, and the actuation zone already consumes commands with a reason, which are the two messages a broker would carry.

Operational notes. The service is started with `make run`, which creates a virtual environment, builds the panel and serves it. Configuration is live: speed, thresholds, dwells, sensor noise and population change on the next tick through `PUT /api/config`, and a reload re-reads the floors from BuildSim without restarting. If BuildSim is down at startup the service says so in its journal and idles until a reload; if it goes away mid-run the service marks itself disconnected, stops writing, and keeps simulating.

10. Test plan

Two kinds of evidence back the requirements. The unit suite, 162 tests and 98 subtests in 22 files, runs in three seconds with nothing else up, because the domain imports nothing that needs a network and the model detector is tested against a fake endpoint and a fake clock. The experiments (E1 to E7) drive the real engine tick by tick against the loaded building with publishing switched off, from the same seed every time, and write their measurements to `experiments/results/`. E7, the model comparison, is the one that also needs the endpoint and a wall clock. Section 11 reports them.

ID	Test	Requirement	Evidence
T-1	`test_agent`: the ladder climbs one step at a time, honours every dwell, and only CONFIRMED and CLEARING issue commands	FR-01, FR-04	unit, passes
T-2	`test_detector`: P(fire) equals the sum of its contributions; no opinion with no coverage; cooking scores under the confirm threshold	FR-03, FR-06	unit, passes
T-3	`test_interlocks`: water refused under 35 °C; lock always refused; close refused when occupied or on a route	SR-01 to SR-03	unit, passes
T-4	`test_sensing`: lag, quantisation, clamp, and each of the four faults behave as specified	NFR-02	unit, passes
T-5	`test_physics`, `test_sources`: relaxation, the diffusion cap, sprinkler factors, the t^2 curve	Section 6	unit, passes
T-6	`test_evacuation`, `test_walkways`, `test_multifloor`: clear routes avoid danger, upper floors reach a ground exit, the drawn route starts at the fire	FR-05	unit, passes
T-7	`test_scoring`, `test_history`: catch, miss and false alarm rules; RSET milestones; the recorder's columns and length	FR-07	unit, passes
T-8	`test_layering`: the import direction and the truth boundary	NFR-04	unit, passes
T-9	`test_publishing_route`, `test_publisher_doors`: the route is sent only when it changes and names its storeys; closed doors reach BuildSim	NFR-05	unit, passes
T-10	`test_reset`, `test_presets`: a reset returns to a quiet building and forgets injected faults; every preset is well formed	NFR-03	unit, passes
T-11	`test_llm_detector`: the prompt carries readings only; a good reply becomes a verdict and a bad one is refused; the rule answers until the model has spoken and again once a verdict expires; a quiet room costs no question; one question per room at a time, most suspicious first; the speed limit follows measured answer times	FR-06, FR-08, NFR-06, NFR-07	unit, passes
T-12	`test_pacing`: the rule has no speed limit; a slow detector caps the dial once and says so; switching back lifts the cap	NFR-07	unit, passes
E1	Each of the six presets in A3505 for 20 simulated minutes: state times, score, response, peak truth	FR-01 to FR-04, SR-01	Table 10, Figure 9
E2	The office fire with the agent's commands dropped (supervised mode), then with the sprinkler alone	Section 6 loop	Table 11
E3	Office fire and kitchen cooking at sensor noise 0, 1, 2, 3×	FR-03, NFR-03	Table 12
E4	Office fire with the smoke detector stuck, dropping out, drifting, or dead	NFR-02	Table 13
E5	Sensors in 10 to 956 rooms; wall time of one tick at 1× and 40×	NFR-01	Figure 10
E6	Every experiment rerun from the same seed	NFR-03	identical latencies to the second
E7	The six presets, cooking at 2 and 3× noise, and the fire with a stuck smoke detector, scored by the language model at the pace it can sustain	FR-08, NFR-06, NFR-07	Table **??**, Figure **??**

11. Evaluation and experiments

This section reports what was measured. Every number below was measured by the experiment harness in `experiments/run_experiments.py` on the loaded A-building, with the fire in office A3505 on the top floor, 130 people scattered by the default seed, and the default thresholds. The room and its neighbours were instrumented with all three modalities, as the presets do.

Detection and response (E1).

Preset	Investigating	Confirmed	Max P(fire)	False alarms	Sprinkler	RSET	Verdict
Office fire (flaming, fast)	51 s	106 s	0.994	0	on at 106 s	217 s	caught ✓
Overnight smoulder	136 s	201 s	0.991	0	refused 201 s, on 219 s	312 s	caught ✓
Kitchen cooking	326 s	never	0.688	0			correctly ignored ✓
Dust storm	never	never	0.129	0			correctly ignored ✓
Fire, dead smoke detector	61 s	never (room); 156 s (neighbour)	0.748	0	on next door	267 s	caught by the neighbour, see below
Drifting sensor, no fire	never	never	0.057	0			correctly ignored ✓

Table 10: *One run of each preset. Times are seconds since ignition. RSET is ignition to the last of 130 people out; in every caught case the building cleared 111 s after the alarm.*

Both fires were confirmed well inside the 180 s and 300 s limits of FR-01 and FR-02, and none of the three nuisances raised an alarm (FR-03). Figure 9 shows the office fire as the system saw it. The truth and the reading agree until the heat detector saturates at 250 °C; the probability climbs through the three thresholds with the dwells visible as the flat time between crossing a level and changing state; and after the sprinkler releases at 106 s the true smoke bends down while the temperature, in a now sealed room, keeps rising. The smoulder is the interlock's showcase: confirmed on CO with the room at 34 °C, its sprinkler request was refused for lack of heat and journalled, then honoured 18 s later when the room crossed 35 °C. The design worked as intended, but the number deserves a second look: 35 °C is a low bar for "heat corroboration", and a warm summer office would clear it.

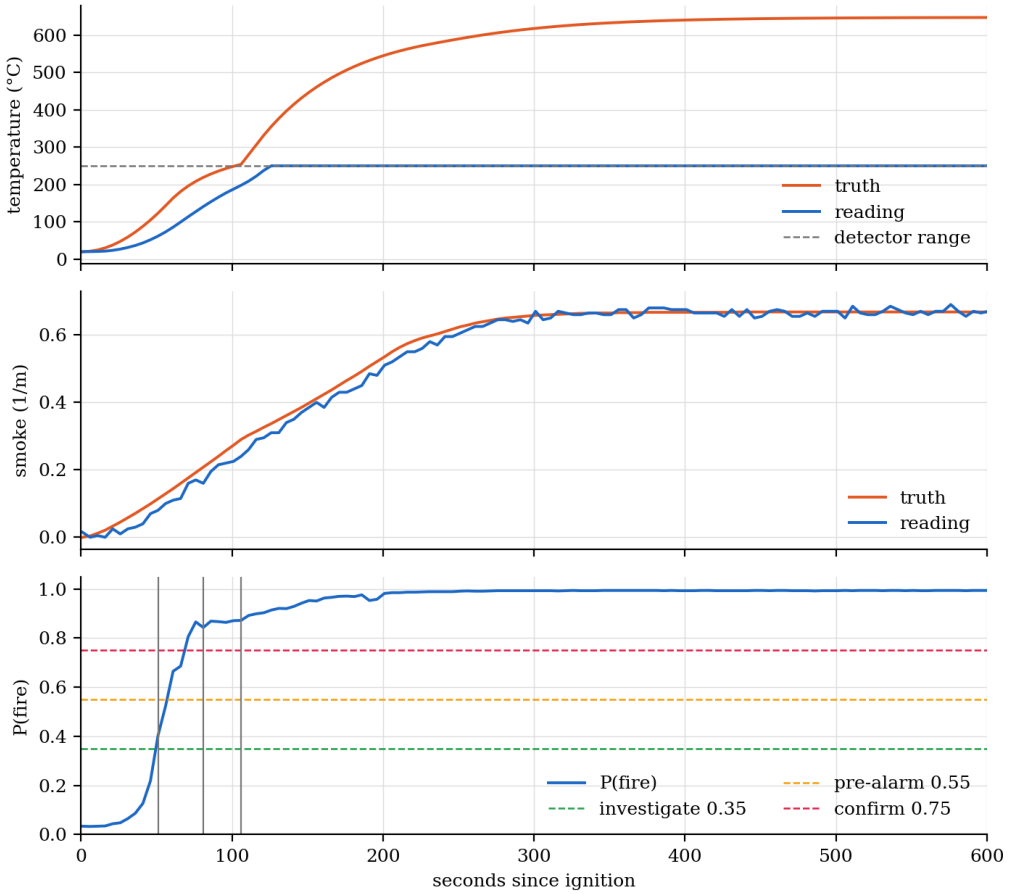


Figure 9: *The first ten minutes of the office fire in A3505 as the system saw it (E1). Top: the heat detector saturates at 250 °C while the room reaches 648 °C. Middle: smoke, where the reading tracks the truth with a 12 s lag. Bottom: P(fire) through the three thresholds; the vertical lines are the state changes at 51, 81 and 106 s, and the flat stretches before them are the dwells.*

The blind spot. The preset with a dead smoke detector is the one result that did not go to plan. Heat and CO alone pushed the fire room to a probability of 0.748, held there by the coverage factor, which caps a two-modality room at 85 % of its logistic score. The confirm threshold is 0.75. The room sat in PRE-ALARM

for the whole run, three thousandths short. The building was still evacuated at 156 s, because smoke crossed into A3500A next door, whose three working detectors confirmed and whose neighbour-agreement term the fire room fed. Two lessons: the neighbour cross-check is not decoration, it is what caught this fire; and the coverage penalty is tuned a hair too hard. Lowering it from 0.55 to 0.60 at the floor, or letting a neighbour's confirmation lift the room, would have confirmed the room itself. This is the first change I would make.

Closing the loop, and what containment actually does (E2).

Response	Rooms lost	Neighbour peak smoke	Neighbour peak temp	Fire room peak temp	RSET
None (supervised, nobody presses)	5	0.382 1/m	149 °C	309 °C	no evacuation
Autonomous	4	0.171 1/m	152 °C	648 °C	217 s
Sprinkler only, door held open	5	0.171 1/m	152 °C	648 °C	217 s

Table 11: *The same office fire three times. "Rooms lost" are rooms that became untenable by the ISO 13571 limits at any point in 20 minutes.*

The response halves the smoke reaching the neighbours and saves one room. It also makes the fire room hotter, and that surprised me until the journal explained it: with the door open, the fire room dumps heat into five neighbours every step; with the doors closed it keeps it. The sprinkler's 25 % factor does not offset that, because in this model water reduces the heat released and does nothing to cool the room directly. The third row is the more revealing one. Holding the fire room's own door open made no difference at all, because at 231 s the neighbour A3500A confirmed on the smoke that had crossed into it and closed *its* doors, one of which is the same doorway. Containment in this building is a property of every room around the fire, not of the fire room alone. Without any response, nobody is told to leave and the RSET is never recorded, which is the simplest argument for the evacuate command being automatic.

Noise (E3) and lying sensors (E4).

Noise	Office fire		Kitchen cooking	
	confirmed	max P	alarms	max P
0×	106 s	0.994	0	0.447
1×	106 s	0.994	0	0.688
2×	106 s	0.995	0	0.749
3×	106 s	0.996	0	0.741

Table 12: *Sensor noise sweep (E3).*

Smoke detector	Confirmed	Max P
healthy	106 s	0.994
dropout (40 % lost)	101 s	0.995
drift	106 s	0.995
stuck	156 s	0.880
dead	never (neighbour 156 s)	0.748

Table 13: *Faults on the fire room's smoke detector during the office fire (E4).*

Detection latency did not move by a second across the noise sweep, and no nuisance alarmed, which is the dwell times doing their job. The number to worry about is the cooking peak at 2× noise: 0.749 against a confirm threshold of 0.75. It did not confirm, and it could not have without first holding 0.55 for 20 s and then sitting 25 s in pre-alarm, which it never did; but the margin on the probability itself is one noise sample wide. The CO term is what keeps cooking down, so the accurate statement is that this detector separates cooking from fire on CO and time, not on any single reading. On faults, a dropping-out detector was caught slightly faster because the samples it did deliver happened to land on rising edges; a stuck one cost 50 s because the rate term went flat; drift did nothing in twenty minutes; and dead is the blind spot above.

Scaling and the breaking point (E5). The tick loop wakes every 250 ms of real time and must finish before it wakes again. At 1× speed one tick is one physics step over the whole building plus one intelligence pass over the monitored rooms; at 40× it is forty physics steps and one pass.

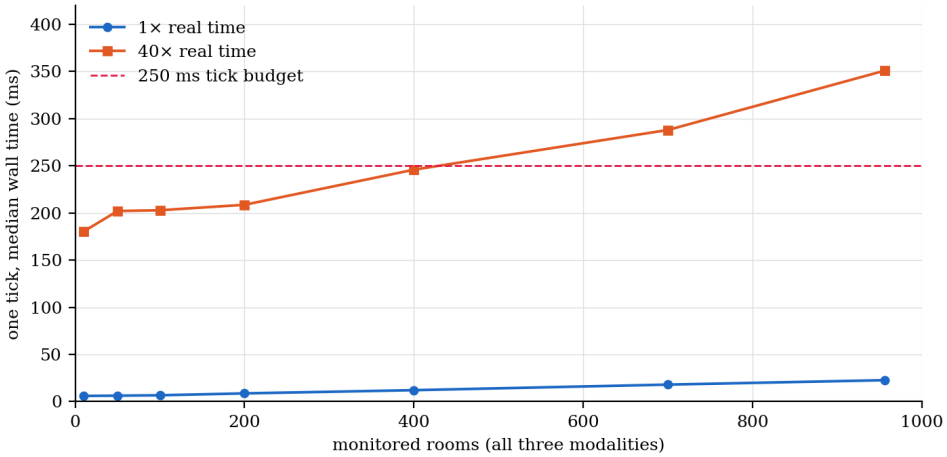


Figure 10: *Wall time of one engine tick against the number of monitored rooms (E5), on the laptop the demo runs on. The building has 956 rooms and 1005 doorways.*

At real time the whole building fits with room to spare: 23 ms per tick with all 956 rooms monitored and 2868 devices sampling. At 40× the budget is crossed at about 450 monitored rooms. The shape says where the cost is. With only 10 rooms monitored a 40× tick already takes 180 ms, so the forty physics steps over the full graph are the fixed cost, about 4.5 ms a step; the intelligence adds about 0.18 ms per monitored room on top. The physics does not scale with the number of sensors, so the fix for a bigger building or a faster clock is a sparser diffusion step, not a faster detector. When the budget is exceeded nothing breaks: the loop runs slower than the speed dial claims, and the tick caps itself at sixty physics steps so a stall cannot snowball.

Headline results.

Metric	Measured	Requirement	
Flaming fire confirmed	106 s	$\leq$ 180 s (FR-01)	✓
Smouldering fire confirmed	201 s	$\leq$ 300 s (FR-02)	✓
False alarms, 3 nuisances × 4 noise levels	0	0 (FR-03)	✓
Commands on confirmation	sprinkler, door, evacuate, same tick, with reasons	(FR-04)	✓
Sprinkler refused without heat	yes, released 18 s later	(SR-01)	✓
Fire caught with a stuck, dropping or drifting smoke detector	yes, worst 156 s	(NFR-02)	✓
Fire room confirmed with a dead smoke detector	no; caught next door at 156 s	(NFR-02)	partial
Tick at 1×, full building	23 ms	$\leq$ 250 ms (NFR-01)	✓
Breaking point at 40×	about 450 monitored rooms	measured (NFR-01)	✓
Building of 130 cleared after the alarm	111 s	reported (FR-07)	✓
Same seed, same run	identical to the second, every rerun	(NFR-03)	✓

12. Dashboard

The dashboard is two browser tabs that are meant to be read together, and each has its own job. The BuildSim viewer shows the building: heat maps of the true temperature, smoke and CO on the floor plates, flames and smoke in the burning room, the sprinkler running, every one of the 130 people walking, the alarming rooms outlined in red, orange or yellow by state, and the one escape route drawn as a red line from the fire room down the stairs to a ground-floor exit. The control panel shows the system: what it is being told, what it believes, and what it did about it.

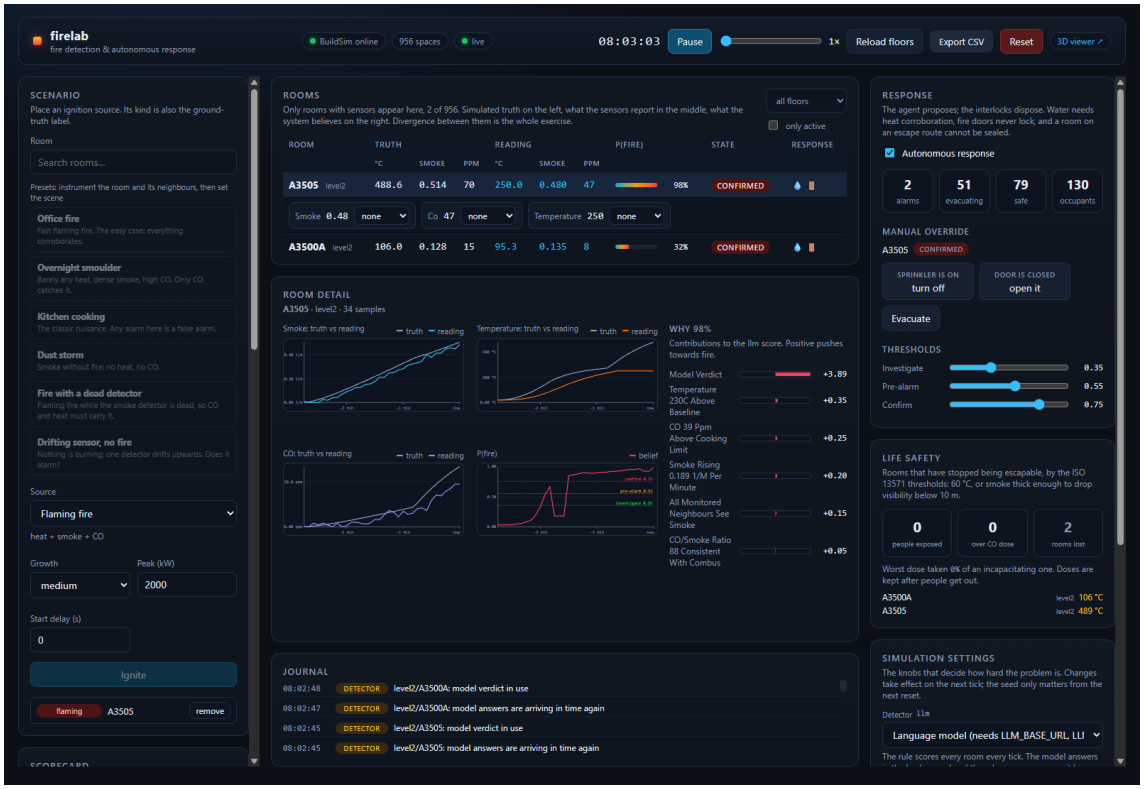


Figure 11: *The control panel during the office fire with the language model as the detector. Rooms table: truth on the left, readings in the middle, P(fire) and state on the right. Room detail: truth against reading for each modality, and the "Why" bars, here the model's own evidence with its verdict as the top bar. Journal: the model's hand-overs. Bottom right: the detector selector.*

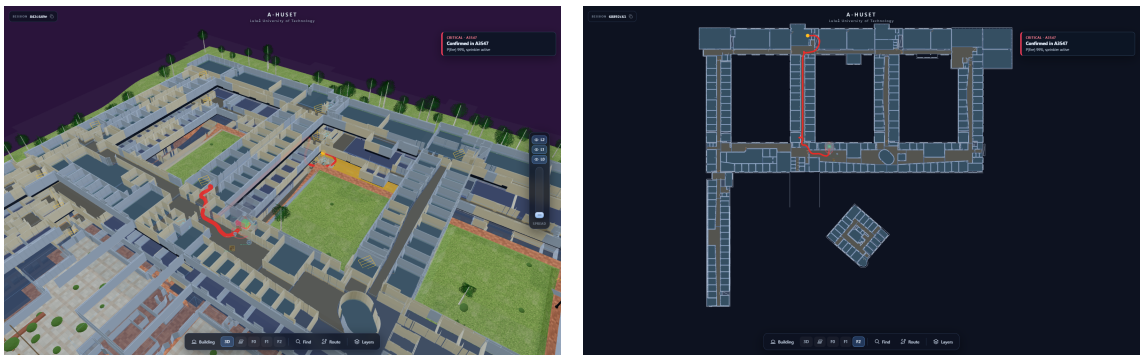


Figure 12: *The BuildSim viewer with firelab's overlays. Left: the 3D view with the fire in the top-floor office and the escape route descending through the stairwell. Right: the same route on the floor-2 plan, from the burning room to the exit.*

The most useful part of the panel is the least visual: the Rooms table places the true value of each modality beside what its sensor reported, and the room detail draws both on one chart. A saturated heat detector, a stuck smoke detector or a drifting one is visible as a gap between two lines, which is the thing an operator of a real building never gets to see and the thing this project is about. Beside it the "Why" bars decompose the probability into its contributions, so a 99 % is not a claim but a sum. With the model installed the same bars show what the model said it weighed, the verdict itself as the top bar, and the heading names which detector is speaking; the P(fire) trace in Figure 11 shows the model's first verdict dragging the room back to normal before its second lifted it, which is the kind of thing a number alone would hide. The journal below records every state change, every command with its reason, every refusal with the interlock's reason, and every hand-over between the model and the rule, and the scorecard grades the run as it happens: catches,

misses, false alarms, the alarm delay and the RSET. The detector is chosen from the settings panel and can be changed mid-fire; the speed dial then shows the cap the model imposes. Everything on the panel arrives over the event stream, one snapshot per tick, and the whole record leaves as a labelled CSV.

13. Risks and critical reflection

The system does what the requirements ask, and the evaluation found four things that deserve more scepticism than a tick mark.

The dead-detector blind spot is real. A room with two of three modalities working stalled three thousandths below the confirm threshold, and was rescued by its neighbour. That is a good outcome in this building, where offices share doorways with corridors, but it depends on the building's layout rather than on the detector. The coverage penalty should be softened, or a confirmed neighbour should be allowed to confirm the room. Both are one-line changes; choosing between them needs more runs than one.

The margins are thinner than the zeros suggest. No nuisance alarmed, but cooking reached 0.749 against a threshold of 0.75. The dwells are what actually kept it quiet. A different kitchen, a bigger pan, or a slightly different noise draw would push the probability over; whether it would then hold for 45 seconds is the real question, and it was not measured across enough seeds to say. The false-alarm rate reported here is zero in the runs performed, not zero.

Containment is modelled too kindly to the neighbours and too harshly to the fire room. Closing the doors halved the smoke next door, which is right, and raised the fire room from 309 to 648 °C, which is a consequence of a model in which water only reduces heat release and a sealed room cannot run out of oxygen. In a zone model the sprinkler would cool the gas layer and the fire would starve. The relative comparisons in Section 11 stand; the absolute temperatures should not be quoted. The sprinkler interlock's 35 °C bar is also low, and the smoulder cleared it in 18 s; a rate-of-rise condition would be more defensible.

One process is the right call for a simulator and the wrong one for a building. The proposal wanted nine containers and the report has one. The justification, one simulated clock, is genuine, and the layering test keeps the zones separable. But nothing here demonstrates fault isolation, and a fire safety system that dies with its physics engine would not be certified. The migration path is known and narrow: the sensing zone already emits per-device readings and the actuation zone already consumes reasoned commands, which are the two topics a broker would carry, and simulated time would travel as a field in every message. The evacuation model is optimistic in the same direction: 130 people out in 111 s is a walking-speed number with no queueing at doors and no one who hesitates.

Risk / assumption	Impact	Mitigation / decision
Coverage penalty caps a two-modality room below confirm	A fire with a dead smoke detector is caught only through its neighbour	Soften the floor of the coverage factor, or let a confirmed neighbour confirm the room; measure over many seeds
Nuisance probability within 0.001 of the threshold	A false alarm under a different noise draw	Dwell times hold; add a multi-seed false-alarm rate to the harness before tuning
Lumped physics with no oxygen limit and no direct cooling	Sealed fire rooms overheat; absolute temperatures flatter or damn the response	Read results relatively; a two-layer zone model is the next physics step
Heat corroboration at 35 °C	Water released on a warm smoulder or a hot day	Require a temperature rise rate as well as a level
Single process, no fault isolation	A crash in one zone stops detection	Acceptable on the laptop; the zone seams are the broker topics of the split
Evacuation without queueing or hesitation	RSET is a lower bound	Report it as such; add door capacity and a pre-movement delay per role
Hand-tuned detector weights	May not transfer to other buildings or sensor mixes	The CSV export and the Detector protocol exist for a trained replacement; the rule is its baseline
Route drawn to one viewer tab, sent once	A second tab, or a reload at the wrong moment, sees no line	Re-send on session change is in; a per-tab replay on connect would finish it

If I had another two weeks they would go, in order, to a multi-seed false-alarm rate, the coverage fix with that rate as its test, and the trained detector the proposal promised, judged against the rule on the same recorded runs and on whether its "Why" can be drawn as bars. The system as it stands is small, explainable, and measured, and the places where it is weak are written down here rather than discovered in the demo.